

Gray-box en Wilson–Cowan

Qué hicimos, qué salió, y qué significa

La idea en una frase. Hasta ahora identificábamos los parámetros de Wilson–Cowan con un modelo que era *exactamente* igual al simulador que generaba los datos. Le rompimos esa igualdad a propósito, y medimos qué pasa cuando le agregamos una red neuronal para tapar el agujero.

1. Glosario, primero

Los términos que van a aparecer todo el tiempo.

Planta — El sistema real. En nuestro caso, el simulador. Es lo que genera los datos.

Modelo — Nuestra hipótesis sobre cómo funciona la planta. Es lo que entrenamos.

Mismatch — El hueco entre los dos: física que la planta tiene y el modelo no. Se escribe Δf . *Mismatch estructural* significa que le erramos a la *forma* de la ecuación, no a un número: ningún ajuste de parámetros lo arregla.

White-box — Modelo hecho sólo de ecuaciones escritas por nosotros. Es el enfoque de siempre del proyecto.

Gray-box — Ecuaciones nuestras *más* una red neuronal que tapa lo que las ecuaciones no explican. Gris = mitad blanco (interpretable), mitad negro.

θ (theta) — Los 10 parámetros de Wilson–Cowan: pesos sinápticos, umbrales, constantes de tiempo. Es lo que queremos identificar, y tienen significado biológico.

g_ϕ — La red neuronal que hace de corrección. Un MLP chiquito: 2 capas, 32 neuronas ocultas.

ϵ (épsilon) — La perilla que controla cuánto mismatch hay. $\epsilon = 0$: ninguno. $\epsilon = 1$: el nivel nominal.

P, Q — El estímulo externo que le mandamos a cada población (P a la excitatoria, Q a la inhibitoria).

MSE — Error cuadrático medio. Mide qué tan bien el modelo reproduce las trayectorias. Es el error de *ajuste*.

R^2 — Mide qué tan parecidas son dos funciones. 1 = idénticas, 0 = tan bueno como predecir el promedio, negativo = peor que el promedio.

2. El problema que teníamos

El proyecto identifica los 10 parámetros con un error del 1 %. Excelente número — pero con trampa.

El simulador que genera los datos y el modelo que entrenamos eran **exactamente las mismas ecuaciones**. O sea que el problema que resolvíamos era: “*dado que el modelo es perfecto, encuentrale los parámetros*”. Legítimo y difícil, pero no es el problema real. En un experimento de verdad tu modelo del cerebro siempre está incompleto.

Y tenía una consecuencia concreta en el código: el término de corrección g_ϕ existía desde el principio y nunca se había usado. No por olvido: **porque no tenía nada que aprender**. Sumarle una red a un modelo que ya es exacto sólo puede empeorar las cosas.

3. Qué hicimos: le rompimos la igualdad

Le agregamos al **simulador** dos pedazos de física que el modelo no tiene. Importante: al simulador, no al modelo. La red nunca ve estas ecuaciones, no sabe que existen.

Perturbación 1 — Refractariedad

$$dI/dt = (1/\tau_i) \cdot (-I + (1 - r \cdot I) \cdot S_i(u_i) - k_i)$$

S es la **sigmoidea**: la función con forma de S que convierte “cuánta entrada recibe la población” en “qué fracción de neuronas dispara”. El factor nuevo dice que una neurona que acaba de disparar no puede volver a disparar enseguida, así que la fracción disponible no es 1 sino $(1 - r \cdot I)$.

No lo inventamos. Es el Wilson–Cowan original de 1972. La versión reducida que usa todo el mundo lo descarta por simplicidad. Nosotros lo *deshicimos*, que es mucho más defendible que inventar una perturbación arbitraria.

Perturbación 2 — El actuador optogenético

$$dP_{\text{lag}}/dt = (P_{\text{cmd}} - P_{\text{lag}}) / \tau_{\text{act}} \quad P_{\text{ef}} = A \cdot \tanh(P_{\text{lag}} / A)$$

En optogenética vos comandás *intensidad de luz*. Lo que la neurona recibe llega tarde (el canal tarda en abrirse) y saturado (a mucha luz ya no responde más). El modelo asume que son iguales.

Ésta es la difícil: rompe el supuesto de que el estímulo es perfectamente conocido, que es justo el supuesto que ancla la escala de los parámetros.

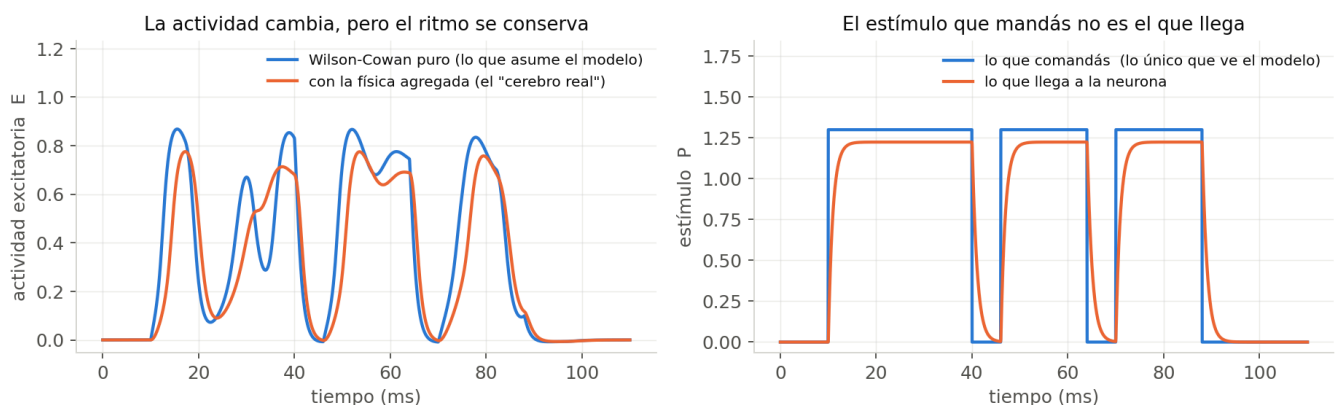


Figura 1. Las dos perturbaciones, en datos reales. *Izquierda:* la actividad del “cerebro real” (naranja) contra la que el modelo espera (azul). El ciclo se deforma pero no desaparece: sigue oscilando con el mismo ritmo, que es lo que queríamos — un mismatch que moleste sin destruir el régimen. *Derecha:* el estímulo comandado (azul, escalones limpios) contra el que realmente llega a la neurona (naranja, redondeado por el retardo y achatado por la saturación). El modelo sólo conoce el azul.

¿Por qué no le agregamos estos términos al modelo y listo? Porque en un experimento real no sabés qué te falta — si lo supieras, lo agregarías. Todo el montaje existe para estudiar qué pasa cuando *no* lo sabés. Y como nosotros fabricamos el simulador, conocemos el Δf exacto y podemos hacer trampa *sólo para evaluar*: preguntarnos si la red aprendió la física correcta. Casi ningún trabajo de este tipo puede responder eso.

4. Cómo quedó el modelo

$$\dot{x} = f_{WC}(x, P, Q; \theta) + g_\phi(x)$$

backbone: nuestras ecuaciones, 10 parámetros interpretables + **corrección:** la red, no interpretable

La trampa que esperábamos

La red es un aproximador universal sobre el mismo dominio que las ecuaciones. Entonces **compite** con los parámetros: para cualquier θ equivocado existe una red que compensa. El síntoma esperado era traicionero: *el ajuste mejora y los parámetros empeoran*. Por eso siempre reportamos las dos métricas juntas.

5. Los resultados

1 · Sin corrección, el mismatch destruye los parámetros

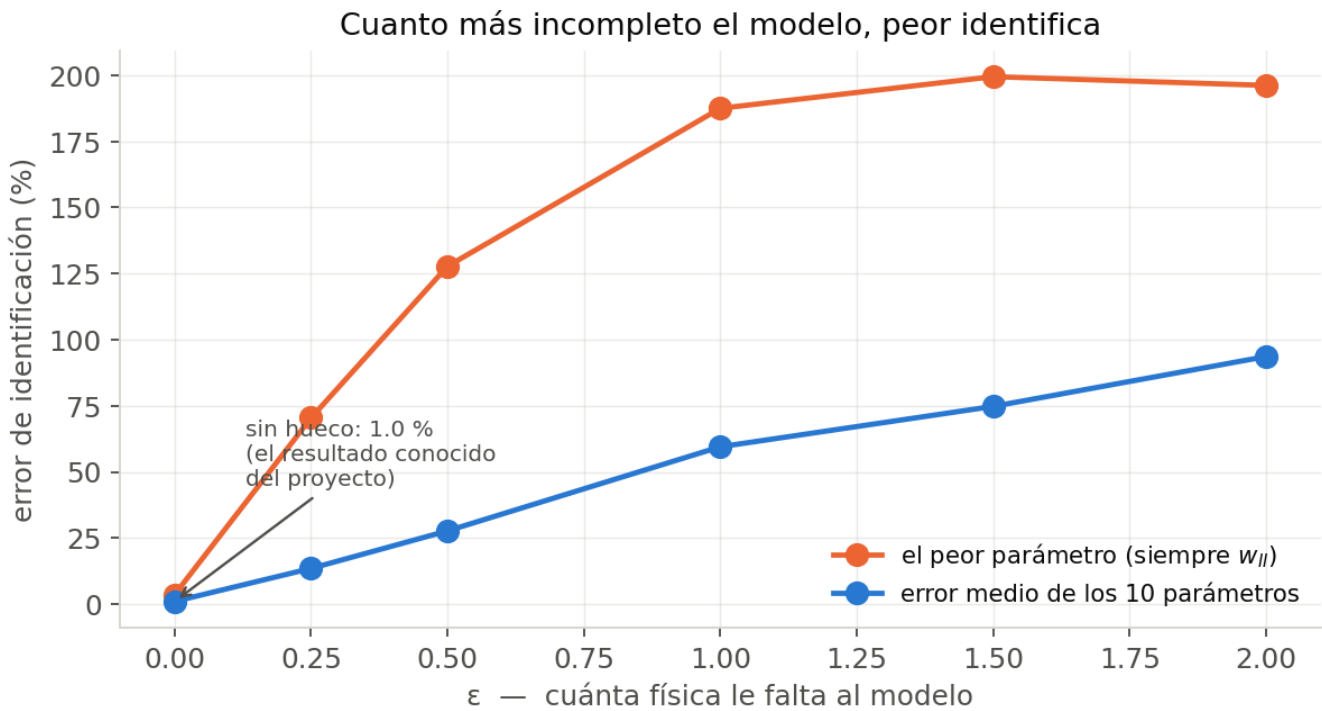


Figura 2. Cuánto cuesta que el modelo esté incompleto. Con $\epsilon = 0$ se reproduce el resultado conocido del proyecto (1 %), así que la maquinaria nueva no rompió nada. A partir de ahí la identificación se degrada rápido. Y **el peor parámetro es siempre w_{II}** , en los seis niveles: exactamente el que el análisis de identificabilidad del proyecto ya señalaba como el más frágil. Confirmación independiente, por un camino totalmente distinto.

2 · El hallazgo que explica todo: dos tercios del hueco se disfrazan de parámetros

Proyectamos el Δf verdadero sobre lo que un cambio de parámetros podría hacer. En $\epsilon = 1$: **el 67 % de la física faltante es indistinguible de mover los 10 parámetros**.

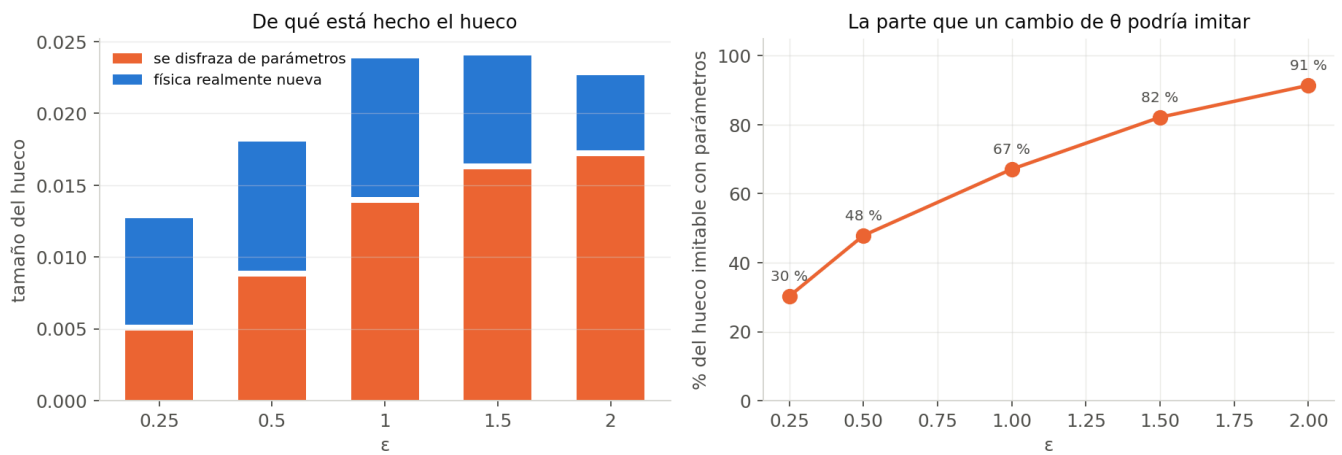


Figura 3. De qué está hecho el hueco. La parte naranja es la que un cambio de θ podría imitar; la azul es física que *ningún* ajuste de los 10 parámetros puede dar. Cuanto más grande el mismatch, más se parece a mover parámetros (derecha) — o sea que el problema empeora en las dos direcciones a la vez.

Esto reordena todo:

- El sesgo del white-box *no es un fallo del optimizador*. Es la mejor respuesta que un modelo rígido puede dar: hay una dirección de parámetros que imita el 67 % del hueco y el ajuste la encuentra. Los 187 % de error en w_{II} son el precio de la rigidez.
- Le pone un techo al gray-box: sobre esa fracción, el reparto entre θ y la red **no es único**. Cualquier división ajusta igual de bien.

3 · Encender la corrección: mejora las dos métricas

$\epsilon = 1$	error de θ	MSE test	redundante con θ
white-box (sin red)	59,5 %	1,06e-2	—
red libre	35,9 %	8,51e-3	0,918
red restringida al estado	35,6 %	9,63e-3	0,941
+ regularización suave	30,8 %	9,26e-3	0,914
+ regularización fuerte	44,6 %	1,06e-2	0,385

La predicción de la trampa era equivocada. Esperábamos que mejorara el ajuste y empeorara los parámetros. Pasó lo contrario: mejora las dos cosas, de 59,5 % a ~31 %. Se recupera casi la mitad del daño.

¿Por qué? Sin la red, el white-box está *obligado* a deformar θ para absorber física que no puede representar. Darle a esa física otro lugar donde ir libera a los parámetros.

4 · Pero no aprendió la física

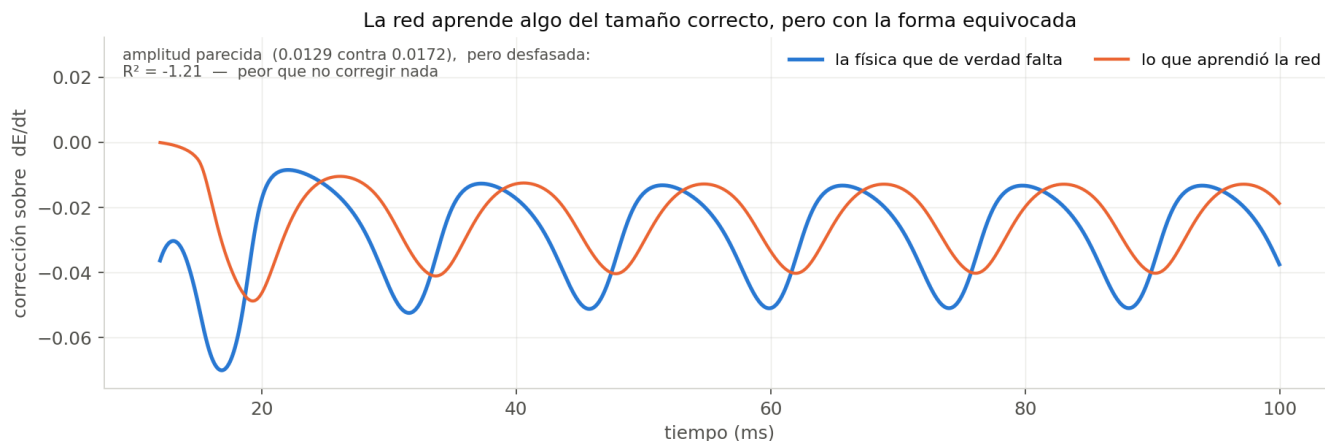


Figura 4. Lo que la red debería haber aprendido (azul) contra lo que aprendió (naranja). Le acierta al tamaño y le erra a la forma: va desfasada. El R^2 negativo significa que, como estimación de la física faltante, es peor que no corregir nada. Y ~90 % de lo que hace lo podría haber hecho un cambio de parámetros.

O sea: ayuda, pero no por la razón que uno querría. No está aprendiendo la física faltante — está haciendo un reacomodamiento parecido a mover parámetros que resulta beneficioso.

5 · El control decisivo: ¿y si no hay nada que corregir?

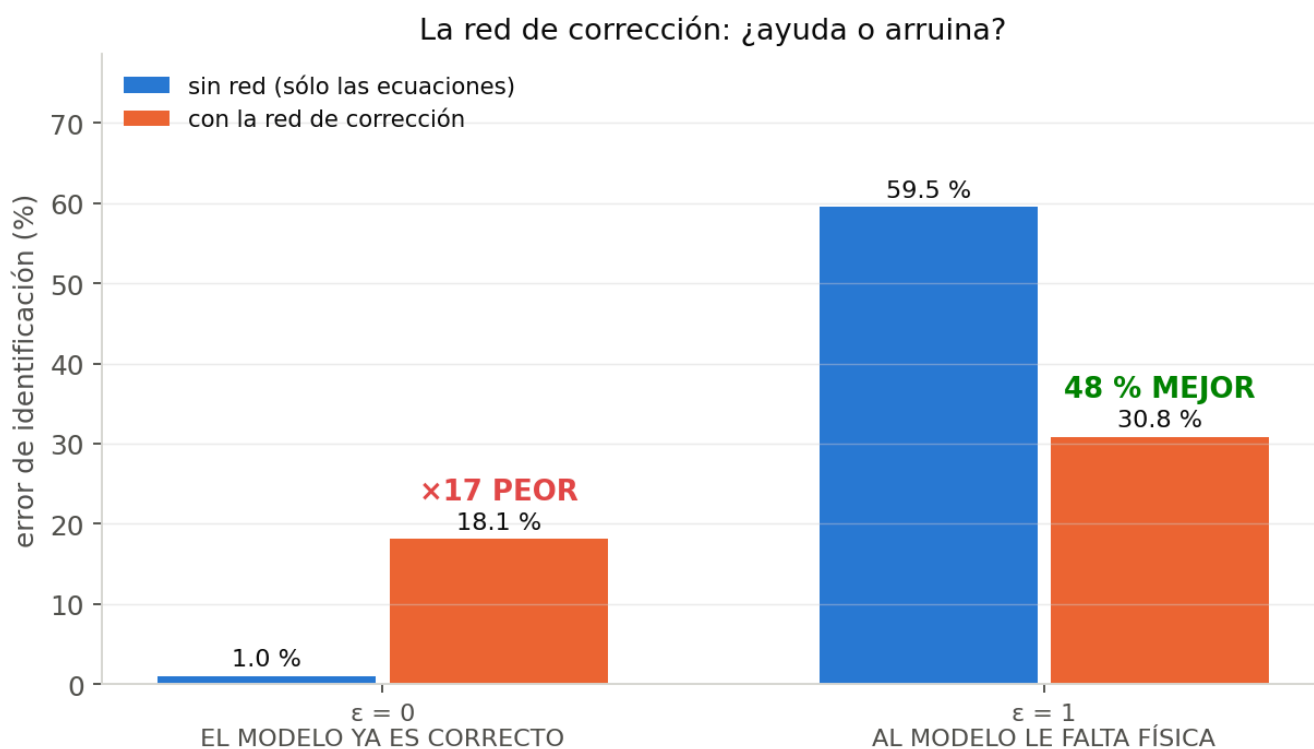


Figura 5. El resultado central del trabajo. Cuando al modelo le falta física, la red ayuda (48 % mejor). Cuando el modelo ya es correcto, la misma red **arruina la identificación 17 veces**.

Una corrección neuronal es dañina justo cuando tu modelo ya es correcto, y útil justo cuando no lo es. Y el problema práctico: **mirando el ajuste no podés distinguir en cuál de los dos casos estás**, porque en $\epsilon = 0$ el MSE mejora igual. Hace falta una métrica que no dependa del ajuste.

6 · ¿Copia bien el comportamiento?

Todo lo anterior mira los *parámetros*. Ésta es la otra pregunta: si el modelo aprendido, soltado a rodar solo, reproduce la actividad del cerebro simulado. La prueba es exigente — se le da **sólo el arranque y el estímulo**, y tiene que generar los 200 ms enteros por su cuenta, sobre estímulos que no vio al entrenar.

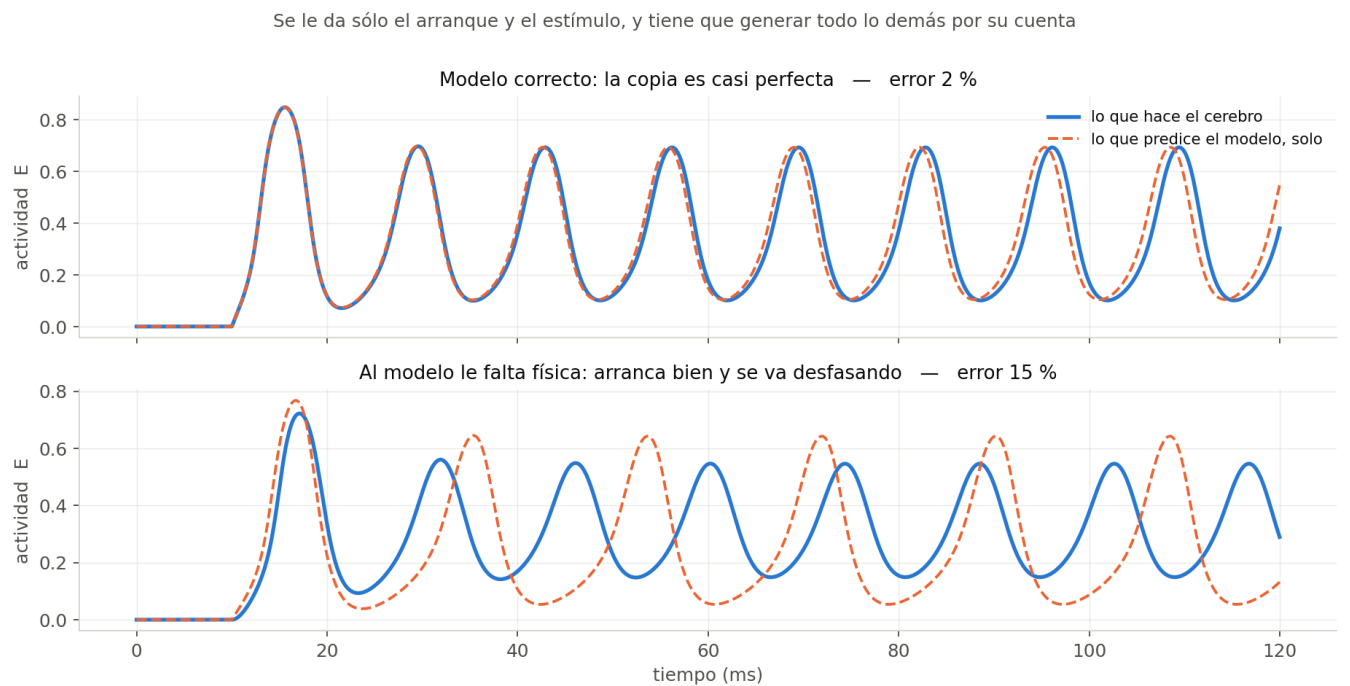


Figura 6. Arriba: con el modelo correcto la copia es casi perfecta (2 % de error después de 200 ms rodando sola). Abajo: al modelo le falta física, arranca bien y se va desfasando — predice una oscilación más lenta que la real. Sirve para entender el sistema; no para predecir con precisión.

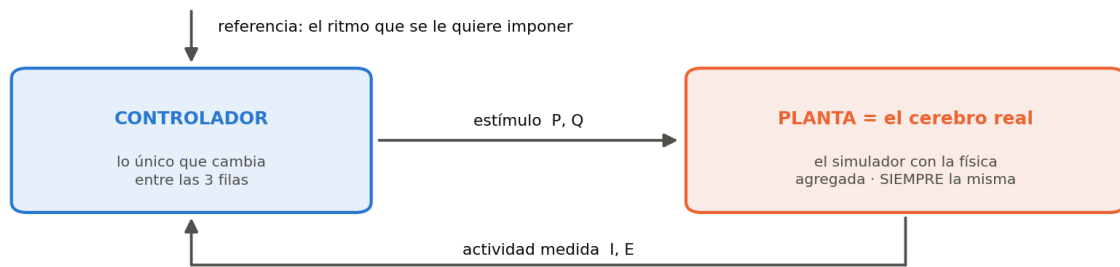
situación	error de trayectoria	correlación
sin hueco (modelo = planta)	2,0 %	0,99
con hueco, white-box	15,1 %	0,82
con hueco, gray-box	14,0 %	0,85

El dato más informativo: la corrección recupera casi la mitad del error de los *parámetros* y casi nada del error de *trayectoria* (15,1 % → 14,0 %). Ayuda a **identificar**, no a **predecir**. Es consistente con lo anterior: mueve la responsabilidad de sitio sin agregar física.

7 · Control: los parámetros sirven, la red no

Primero, quién es quién — porque acá es fácil confundirse. En este experimento **la planta es siempre la misma**: el simulador real con la física agregada, o sea el “cerebro”. **El modelo aprendido no reemplaza al cerebro**. Lo único que cambia entre las tres filas es *con qué conocimiento se arma el controlador*.

Quién es quién en el lazo de control



Lo único que cambia entre las tres filas de la tabla es qué se le pone adentro al controlador:

- white-box**
los 10 parámetros identificados sin usar la red
- gray-box, sólo los parámetros**
los 10 parámetros que salieron de entrenar CON la red — y después la red se descarta
- gray-box, restando la red**
lo mismo, y además evaluar $\hat{g}(I, E)$ en cada paso y restarlo del objetivo

Figura 7. El lazo. El controlador mide la actividad, la compara con la referencia y decide qué estímulo mandar. Para decidirlo necesita un modelo del cerebro — y ahí es donde entran (o no) las cosas que salieron del entrenamiento. *(En otras partes del proyecto sí se usa la Neural ODE como planta, para ver si el modelo aprendido puede hacer de sustituto. Acá no: acá se la evalúa contra el cerebro real, que es la prueba más exigente.)*

¿Qué significa “usar los parámetros” y “restar la red”?

El controlador cancela la no linealidad del cerebro invirtiéndola. Hace tres pasos: calcula cuánto tiene que dar la sigmoidea, la invierte para saber cuánta entrada total necesita la población, y le resta el acoplamiento interno para saber qué estímulo externo comandar. **Los 10 parámetros aparecen en esos tres pasos** — sin ellos no puede hacer ninguno.

Usar sólo los parámetros — El controlador se arma con los θ que salieron del entrenamiento *gray-box*. Son mejores que los del *white-box* porque durante el entrenamiento la red absorbió parte del hueco y liberó a los parámetros. Pero a la hora de controlar, **la red se tira**: no participa. El *gray-box* se usó como andamio.

Restar también la red — Además de eso, en cada paso se evalúa $\hat{g}(I, E)$ y se le resta al objetivo. Sale de una cuenta corta: si tu modelo dice que la derivada es $(1/\tau)(-I + S - k) + \hat{g}$ y querés que el resultado sea $(1/\tau)(-I + U_{\text{lti}})$, despejando queda $S = U_{\text{lti}} + k - \tau \cdot \hat{g}$. O sea: alcanza con correr el objetivo de la sigmoidea por $-\tau \cdot \hat{g}$. Es una línea de código, y en teoría compensa exactamente el término extra.

Para controlar: los parámetros sirven, la red no

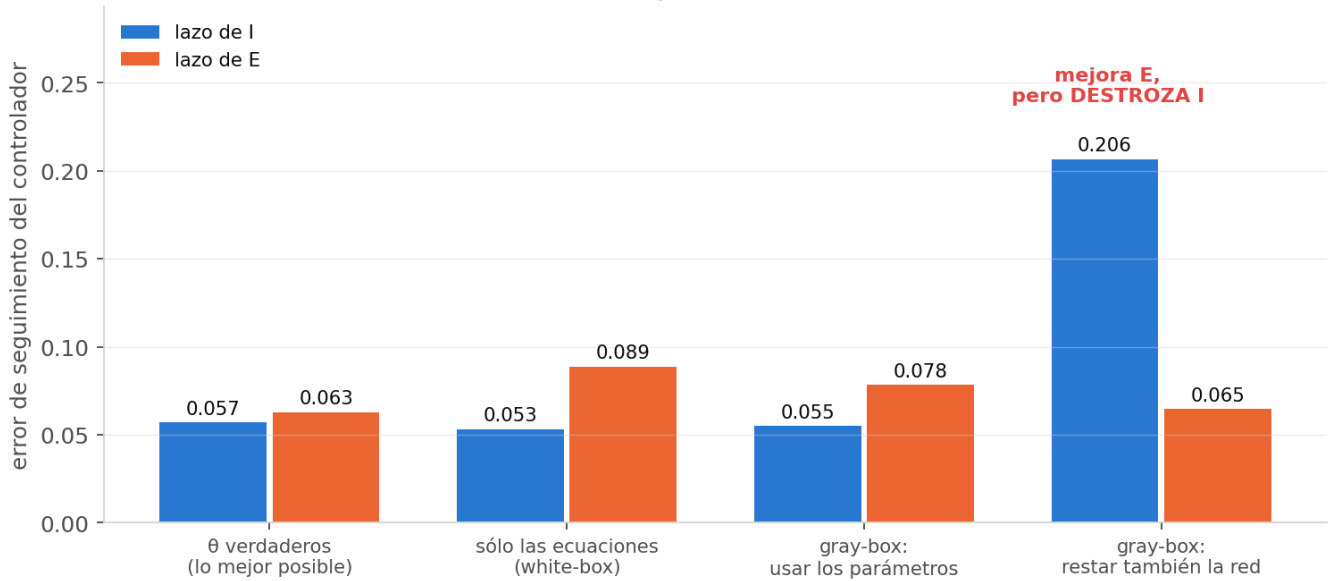


Figura 8. Lo que pasa en la práctica. Usar los parámetros del gray-box mejora respecto del white-box (0,089 → 0,078 en E). Restar la red suena bien y **no funciona**: como acierta la amplitud pero va desfasada, en muchos instantes le resta con el signo equivocado. Arregla el lazo de E y destroza el de I (0,055 → 0,206). Hay que mirar los dos lazos: mirando sólo E parecería que mejora.

La receta: usá los parámetros del gray-box, no uses la red para cancelar. Y el motivo de fondo: la resta es *punto a punto*, así que necesita que $\hat{g}$ sea la física faltante *de verdad*. Que ajuste bien los datos globalmente no alcanza — son dos requisitos distintos.

6. Las tres formas de usar esto

Después de entrenar tenés dos cosas: los **10 números** y la **red**. Qué hacés con ellas depende del objetivo.

Si querés...	Hacés...
Los parámetros, porque significan algo biológico	Entrenás con la red prendida, te quedás con los números, tirás la red. La red fue un andamio. Funciona
Predecir trayectorias	Dejás la red puesta: el modelo completo predice apenas mejor. Ojo: los números pueden estar arruinados y no te enterás.
Controlar el sistema	Usás los números, tirás la red. Restar la red rompe un canal.

7. Lo que hicimos después

Tres frentes que quedaron abiertos y ya se cerraron o avanzaron.

El integrador: absuelto

Apareció que el integrador que usan los scripts del proyecto acumula un error de $\sim 1,5e-2$, del mismo orden que el ruido de los barridos de robustez ($\sigma = 0,01$). Como es un error *sistemático*, promediar no lo elimina — había que descartar que estuviera contaminando un resultado ya establecido.

integrador	error de θ con $\sigma=0$	error de θ con $\sigma=0,01$
histórico (el que se usa hoy)	0,54 %	8,14 %
paso fijo (30× más preciso)	1,05 %	8,39 %
referencia de alta precisión	1,07 %	8,41 %

No contamina. Al nivel de ruido que importa los tres coinciden dentro del 3 %. **Los barridos de robustez son válidos y no hay que rehacerlos.**

Diseño de estímulos: se puede romper la degeneración

El 67 % imitable no es una constante: depende de la trayectoria, y la trayectoria la fija el estímulo. Optimizando la forma del estímulo **a igual amplitud media**, baja del 71 % al **59,5 %**.

Pero apareció un matiz que cambia la receta. Un dataset con 20 variantes de ese único estímulo diseñado da 73,9 % — peor que el de librería (67,1 %). La razón es instructiva: cuando se exige un único cambio de parámetros que explique *todos* los escenarios a la vez, la degeneración se rompe sola si los escenarios son **distintos entre sí**. La diversidad pesa más que la calidad de cada uno.

Hay que diseñar el **experimento entero**, no la señal: elegir un conjunto de estímulos cuyas degeneraciones no se solapen. Es diseño de experimentos con una función objetivo nueva — separar la red de los parámetros, en vez de sólo informar sobre los parámetros.

Corrección estructurada: física en vez de caja negra

Como la red no sirve para cancelar, probamos reemplazarla por **3 parámetros con significado físico** (refractividad y saturación) en vez de ~1000 pesos. Ventaja: la cancelación en el controlador vuelve a ser exacta.

Falló, y el diagnóstico es lo interesante: el optimizador encuentra un ajuste **mejor que el de la física verdadera** con parámetros equivocados. ¿Por qué? A la corrección le falta una componente — el *retardo* del actuador — y usa las que sí tiene para compensar la que le falta.

La confirmación es limpia: sobre una planta que tiene *exactamente* la forma que la corrección puede representar, recupera la saturación con 1,3 % de error y la pérdida cae 145 veces.

La corrección estructurada es **identificable y precisa si la forma está completa**, y engañosa si le falta una pieza. Es la misma lección de antes, un nivel más abajo. Advertencia práctica: un gray-box estructurado que ajusta bien **no garantiza** que los parámetros físicos que reporta sean correctos.

8. Qué agrega esto al proyecto

- **El resultado de siempre, ahora defendible.** Pasamos de “identificamos 10 parámetros cuando el modelo es exacto” a “identificamos 10 parámetros bajo mismatch estructural, y medimos cuánto cuesta”.
- **Confirmación independiente del análisis de identificabilidad.** El mismatch degrada primero w_{II} , que ya sabíamos que era la dirección más débil. Y el 93 % de lo que la red le roba a los parámetros va por esa misma dirección.
- **Resultados que, hasta donde vimos, nadie reporta:** el 67 % de mismatch imitable, el cruce en $\epsilon = 0$, que *ajustar bien no habilita a cancelar*, y que identificar y predecir mejoran de forma desacoplada.

Y todos sólo se pueden obtener en un montaje donde el Δf verdadero es conocido — que es exactamente la ventaja de haber construido la incertidumbre a mano en vez de tomarla de datos reales.

9. Qué sigue

1. **Diseñar el conjunto de estímulos** (no uno solo) que minimice la degeneración conjunta, y validar que con él la red *sí* aprenda la física — o sea, que el R^2 se vuelva positivo. Si eso pasa, la historia da vuelta: pasaríamos de “ayuda sin aprender física” a “con el experimento bien diseñado, aprende física”. Es de un día de trabajo y el código ya está escrito.
2. **Completar la corrección estructurada con el retardo.** El diagnóstico ya está hecho: falla *sólo* por esa componente. Sería una corrección de 4 parámetros físicos, identificable, interpretable y con cancelación exacta — el camino más prometedor para el control.
3. **Cuando esto vaya a datos reales:** ahí no existe el Δf verdadero, así que no hay forma de saber si la red aprendió física o está compitiendo con los parámetros *mirando el ajuste*. Lo mínimo a reportar es la fracción redundante contra una red sin entrenar. Sin eso, un gray-box puede estar destruyendo la identificación y parecer un éxito.

Todos los números salen de corridas reales del repositorio (`results/uncertainty/`), reproducibles con `bash scripts/run_uncertainty_all.sh` . Detalle completo en `docs/graybox_manual_completo.md` . · 38 tests automáticos en verde.